

6.25:

第一行: 64; 24; 6; 2;
 第二行: 1; 30; 0; 2;
 第三行: 128; 22; 7; 3;
 第四行: 1; 29; 0; 3;
 第五行: 32; 22; 5; 5;
 第六行: 8; 24; 3; 5;

6.29:

A:

12~4: CT;
 3~2: CI;
 1~0: CT;

B:

0x834: 不命中; 不命中, 因此读不出值;
 0x836: 命中; 值未知;
 0xFFD: 命中; 0xC0;

6.34:

块大小为 16, 又因为是直接映射, 故一行高速缓存行可以存下数组的一行, 而总共有 32 个数据字节, 故有两行高速缓存行, 两个 set, 对于 dst 数组, 由于 $64\%32=0$, 它的第 0, 第 2 行会缓存到第一个高速缓存行 (第一个 set), 第 1, 第 3 行缓存到第二个高速缓存行 (第二个 set), 对于 src 数组也是同理。

接着按照先读后写的顺序迭代分析即可:

dst: 全是 miss

src	到 0	1	2	3
行 0	m	m	h	m
1	m	h	m	h
2	m	m	h	m
3	m	h	m	h

6.34a:

有 64 个数据字节, 意味着有 4 个 set, 由于 $64\%64=0$, dst 的第 1 行映射到第一个 set, 第 2 行映射到第二个 set。。。同理可得, src 也是。之后同样进行迭代分析即可:

dst					src				
到 0	1	2	3		到 0	1	2	3	
行 0	m	m	h	h	行 0	m	m	h	h
1	m	m	m	h	1	m	h	m	h
2	m	h	m	m	2	m	h	h	m
3	m	h	h	m	3	m	h	h	h

6.36:

A: $512/4=128$; , 有 128 个 set, 故每个循环中, $x[0][i]$ 和 $x[1][i]$ 都映射到同一个 set, 每循环一次, 就一定不命中, 故不命中率为 100%;

B: 缓存翻倍, 每一个循环中, $x[0][i]$ 和 $x[1][i]$ 不再映射到同一个 set 中, 而每个高速缓存块可以存 4 个 int, 故不命中率为 $1/4=25\%$;

C: 两路组相联, 有 64 个 set, $128\%64=0$, 故 $x[0][i]$ 和 $x[1][i]$ 都映射到同一个 set, 只不过此时 $x[1][i]$ 不会替换 $x[0][i]$ 了, 循环到 64 时会从头替换, 不过不影响, 每次循环, $x[0][i]$ 和 $x[1][i]$ 一定各有一次不命中, 故不命中率为 25%;

D: 不能; 高速缓存大小增大, $x[0][i]$ 和 $x[1][i]$ 不再映射到同一个 set, 但第一次的缓存不命中始终存在, 故不会增大;

E: 能, 若块增大两倍, 7 个数据中只有 1 个不命中, 降低了不命中率;

6.41:

显然是 $1/4=25\%$;

这个结构体正好占 4 个字节, 每次读 (写) 必将一个完整的 pixel 放到缓存里, 而每个 pixel 只用一次, 故第一个必不命中, 不命中率为 25%;